AIDCF

AI Data Centre Facilities Assurance Framework

MACHINE-EXECUTABLE DATA MODEL STANDARD

Serialisations, identifiers, schemas, ontology, shapes, canonical rules and conformance tests for AIDCF records

Document control

Field	Value
Public ID	AIDCF-016
Short code	MDM (requirement IDs MDM-REQ-NNN; canonical rule IDs MDM-RNN)
Title	Machine-Executable Data Model Standard
Version	1.0
Schema set	MDM 1.3 (ontology version IRI `https://w3id.org/aidcf/ontology/1.3`)
Date	2026-10-10
Status	Draft for controlled review
Author	Siva Sethumadhavan
Depends on	AIDCF-003, AIDCF-004, AIDCF-007A, AIDCF-008, AIDCF-009, AIDCF-010, AIDCF-011, AIDCF-012, AIDCF-013, AIDCF-014
Normative companion of	AIDCF-014 End-to-End Traceability and Machine-Executable Data Standard (TDS-REQ-045)
Reference implementation	AIDCF-015 v1.3 (`implementation/aidcf-015-v1.3/`, release AIDCF-REL-2026.10.1)
Machine package	`schemas/` (generated from `schemas/tools/aidcf_model.py`)
Boundary	Conformance to this standard is technical conformance of data. It is not engineering approval, assurance or certification of any facility.

Revision history

Version	Date	Summary
1.0	2026-10-10	First issue under the public ID AIDCF-016 (the identifier was previously misapplied to the full-stack validation report, now AIDCF-VR-001). Defines the MDM 1.3 schema set, the canonical decision rules MDM-R01 to MDM-R20, validation layers, conformance classes and the release manifest profile. Decision ranks aligned to AIDCF-010 (NOT_ASSURED below INCONCLUSIVE; tie-break to INCONCLUSIVE removed from §9.4 rule 3 and §9.5); reference decision restated as NOT_ASSURED.

1 Purpose and scope

AIDCF-014 defines the information contract of the framework: the 48 canonical classes (§7), the common envelope (§8), object profiles (§8A), the 41 relationship types (§9), the relationship envelope (§10), traceability rules TR-01 to TR-18 (§11), controlled vocabularies (§11A), business rules BR-001 to BR-015 (§18), canonical

queries (§19), the event model and change-impact contract (§20 to §22) and conformance tests CT-01 to CT-14 (§31). AIDCF-014 TDS-REQ-045 names this standard as its normative serialisation companion.

This standard defines how that contract is serialised, validated and executed:

1. the formats, files and versioning of the machine package (§3);
2. the namespace, IRI and identifier policy (§4 to §6);
3. the class, vocabulary and relationship serialisation, including the supporting classes needed by AIDCF-003 that AIDCF-014 has not yet registered (§7 and §8);
4. the canonical decision rules that every conformant implementation shall compute identically: gate roll-up, scope and NOT_ASSESSED handling, ceiling derivation, decision ceiling, score and change impact (§9);
5. canonical SPARQL queries (§10), validation layers (§11), conformance classes and the test catalogue (§12);
6. the release-manifest profile (§13) and versioning and migration (§14).

It applies to every producer and consumer of AIDCF records: assessment tools, registries, evidence systems, reporting pipelines, continuing-assurance engines and the AIDCF-015 reference implementation.

Where this standard and AIDCF-014 differ, AIDCF-014 governs meaning and this standard governs representation. Where this standard records an interpretation of AIDCF-014 that AIDCF-014 does not state, the interpretation is listed in Annex A so that it can be confirmed or corrected in AIDCF-014.

2 References, terms and conventions

2.1 Normative references

AIDCF artifacts are cited by public ID. External specifications are cited at document level:

Reference	Use in this standard
JSON Schema, draft 2020-12 (json-schema.org)	Structural validation of records
JSON-LD 1.1 (W3C Recommendation)	Mapping of JSON records to RDF
RDF 1.1 and RDF 1.1 Turtle (W3C Recommendations)	Graph exchange and the ontology and shapes files
OWL 2 Web Ontology Language (W3C Recommendation)	Class and property declarations
SKOS Simple Knowledge Organization System (W3C Recommendation)	Controlled vocabularies in RDF
Shapes Constraint Language, SHACL (W3C Recommendation), including SHACL-SPARQL	Closed-world graph validation
SPARQL 1.1 Query Language (W3C Recommendation)	Canonical queries
IETF RFC 8785, JSON Canonicalization Scheme	Canonical form of the release manifest
SHA-256 (FIPS 180-4)	File and record digests
ISO 8601	Dates, times and durations
IEC 62443 series (as referenced by AIDCF-003)	Zone, conduit, security-level and foundational-requirement concepts of the supporting classes

2.2 Terms

Term	Meaning in this standard
Record	A JSON object carrying the common envelope (§5) and representing one instance of one class.
Edge	A forward relationship property on a record whose values are identifiers of target records (§8.1).
Relationship record	A RelationshipRecord instance carrying the AIDCF-014 §10 envelope for exactly one edge (§8.2).
Schema set	The generated files under `schemas/`: JSON Schemas, JSON-LD context, OWL ontology and SHACL shapes.
Model	`schemas/tools/aidcf_model.py`, the single source from which the schema set is generated.
Registry copy	An unmodified copy of a repository registry or vocabulary CSV inside a release package, with its digest.
In scope (gate)	A registry CRITICAL control of a gate whose gate is applicable to the declared boundary (§9.3).
Graph-level constraint	A SHACL-SPARQL constraint whose focus node is the ontology IRI and whose value is the offending record (§11.2).

2.3 Conventions

"Shall", "should", "may" and "can" are used as defined in the canonical specification (ISO/IEC Directives Part 2 usage). Requirements carry identifiers MDM-REQ-NNN, numbered consecutively. Canonical rules carry identifiers MDM-R01 to MDM-R20 and are executable checks; a rule failure is a validation failure of the data, not a requirement on an organisation. Codes are written in `UPPER_SNAKE_CASE` exactly as in `registries/vocabularies/`.

3 Serialisation package

3.1 Formats

Format	File(s)	Role
JSON	Record files; release manifest	Authoritative record form (AIDCF-014 §12)
JSON Schema 2020-12	`schemas/json-schema/*.schema.json`	Structural validation: envelope, profiles, codes, patterns, closed objects
JSON-LD 1.1	`schemas/jsonld/aidcf-context-1.3.jsonld`	Maps every JSON key to an `aidcf:` IRI; records become RDF without loss
RDF/Turtle	Projection of records; ontology and shapes files	Graph exchange and query
OWL 2	`schemas/owl/aidcf-core-1.3.ttl`	Classes, properties, domains, ranges, inverses, vocabularies as SKOS schemes
SHACL	`schemas/shacl/aidcf-shapes-1.3.ttl`	Closed-world cardinality, vocabulary and rule validation
CSV	`registries/**/*.csv`	Registries and controlled vocabularies (normative for codes)

MDM-REQ-001 A conformant producer shall emit records as JSON that validate against the JSON Schema of their class in the schema set pinned by the ReleaseManifest the records declare.

MDM-REQ-002 A conformant consumer shall be able to convert every record to RDF with the pinned JSON-LD context alone, without network access to any external context, and the conversion shall produce no triple outside the `aidcf:` namespace other than `rdf:type`.

MDM-REQ-003 The schema set shall be generated from a single model. A release shall include a check that regenerates the schema set and fails if any generated file differs from the published file (`build_schemas.py --check`).

3.2 Files of the MDM 1.3 schema set

File	Content
`definitions.schema.json`	Common definitions: `iri`, `date`, `dateTime`, one `code-<Scheme>` enumeration per controlled vocabulary, and one `obj-<Name>` definition per nested object (Provenance, Scope, GateMapping, ControlEvaluation, CriterionResult, ScoreInput, Coverage, ManifestEntry and others).
`common-envelope.schema.json`	The envelope properties of §5 with their definitions.
One schema per class	58 closed class schemas: 48 canonical, 3 reference, 6 supporting, 1 auxiliary (§7). File names are the kebab-case class name except where AIDCF-014 §15 implies a shorter name (for example `capability`, `architecture`, `evidence`, `score`, `decision`, `report`, `system`, `program`).
`aidcf-context-1.3.jsonld`	Protected context with `@vocab`, typed references (`@type: @id`), ordered lists (`@container: @list`), dates and numbers typed, extension data as `@json`.
`aidcf-core-1.3.ttl`	OWL ontology, SKOS concept schemes, cardinality and qualifier annotations.
`aidcf-shapes-1.3.ttl`	SHACL node and property shapes and SHACL-SPARQL rule shapes.

MDM-REQ-004 Every class schema shall set `additionalProperties: false`, repeat the full required list of its class (envelope plus profile), carry `x-aidcf-class` (class IRI), `x-aidcf-semantic-owner` (owning artifact) and `x-aidcf-registration` (`canonical`, `reference`, `supporting` or `auxiliary`), and have a versioned `$id` under `https://w3id.org/aidcf/schema/1.3/`.

MDM-REQ-005 Extension data shall be carried only in the `extensions` property, keyed by registered namespace prefix (AIDCF-014 §33), and serialised in RDF as a single `rdf:JSON` literal so that extensions cannot introduce triples in the `aidcf:` namespace.

4 Namespace and IRI policy

Kind	IRI pattern	Example
Ontology terms	`https://w3id.org/aidcf/ontology#<Term>` (prefix `aidcf:`)	`aidcf:GateResult`, `aidcf:evaluatesGate`
Ontology version	`https://w3id.org/aidcf/ontology/<release>`	`https://w3id.org/aidcf/ontology/1.3`

Kind	IRI pattern	Example
Vocabulary concepts	`https://w3id.org/aidcf/vocab/<scheme-kebab>/<CODE>`	`https://w3id.org/aidcf/vocab/gate-state/FAIL`
Shapes graph	`https://w3id.org/aidcf/shapes/<release>`	`https://w3id.org/aidcf/shapes/1.3`
Schemas	`https://w3id.org/aidcf/schema/<release>/<file>`	`.../schema/1.3/gate-result.schema.json`
Registry objects	`https://w3id.org/aidcf/id/<kind>/<code>`	`.../id/control/MCF-THM-010`, `.../id/gate/CG-06`, `.../id/family/THM-03`, `.../id/domain/THM`, `.../id/evidence-requirement/MCF-THM-010-IMPLEMENTATION`, `.../id/foundational-requirement/FR5`, `.../id/operating-state/MAINTENANCE`
Facility records	`urn:<registered-nid>:...` or an `https://` IRI controlled by the record owner	(implementation-specific)
Synthetic records	`urn:aidcf:syn:<kind>:<local>`	`urn:aidcf:syn:gate-result:GR-CG-07`

Registration of the `w3id.org/aidcf` redirect is pending. Until it is confirmed, these IRIs are identifiers only and shall not be assumed to resolve (AIDCF-014 §5.1).

MDM-REQ-006 Registry objects (domains, families, controls, critical gates, evidence requirements, foundational requirements, operating states) shall use the registry-object IRI pattern, with the canonical code as the last path segment, so that every implementation names the same registry object with the same IRI.

MDM-REQ-007 Synthetic records shall use the `urn:aidcf:syn:` namespace, and every synthetic record shall carry `provenance.synthetic = true`. A record in the `urn:aidcf:syn:` namespace shall not be referenced by an AssuranceDecision about a real facility.

MDM-REQ-008 Identifiers of records from earlier releases that do not follow this policy (for example `urn:aidcf:assessment:CA-001` in AIDCF-015 v1.2) shall not be reused for new records; they may appear only as targets of MIGRATED_FROM edges, which may point to identifiers outside the dataset.

5 Common object envelope

Every record carries the AIDCF-014 §8 envelope. The serialised envelope is:

Property	Required	Rule
`id`	Yes	IRI per §4. Unique across the release.
`type`	Yes	Class local name (for example `GateResult`); maps to `rdf:type`.
`semanticOwner`	Yes	Public ID of the artifact that owns the class meaning (`AIDCF-NNN`, `AIDCF-007A`, `AIDCF-0NN-XXX`, `AIDCF-VR-NNN`, `AIDCF-WP-NNN`).
`schemaVersion`	Yes	Constant `1.3` for this schema set.
`version`	Yes	Record version (semantic version), carried separately from identity (TDS-REQ-029).

Property	Required	Rule
`status`	Yes	RecordLifecycle code (AIDCF-014 §11A.1).
`releaseRef`	Yes	`releaseId` of the ReleaseManifest the record is valid against (pattern `AIDCF-REL-YYYY.MM.N`).
`sourceArtifact`	Yes	Public ID of the artifact the record was derived from.
`name`	Yes	Human-readable name (`rdfs:label` sub-property).
`effectiveFrom`, `createdAt`	Yes	ISO 8601 date-time with explicit offset.
`createdBy`	Yes	Reference to a PartyEntity.
`classification`	Yes	Classification code (AIDCF-014 §11A.8).
`provenance`	Yes	Object: `sourceSystem`, `method` (ProvenanceMethod), `synthetic`; optional `transformationVersion`, `legacyValue`, `sha256`.
`reviewState`	Conditional	Required for assertion-bearing classes (§7.5). AssertionState code (AIDCF-014 §11A.2).
`scope`	Conditional	Required for assessable classes (§7.5). Object: `capabilityRef`; optional `snapshotRef`, `boundaryNote`.
`distributionControlRef`	Conditional	Required when `classification` is CLIENT_RESTRICTED, CONFIDENTIAL or HIGHLY_RESTRICTED.
`sourceRecord`, `description`, `effectiveTo`, `modifiedAt`, `modifiedBy`, `externalIdentifiers`, `extensions`, `nullReasons`	No	As AIDCF-014 §8 and §13.
`hasUnknown`, `reviewedBy`, `supersedes`, `migratedFrom`, `integrationHandoff`, `governedBy`	No	Relationships permitted on any record (AIDCF-014 §9 rows with unrestricted domain).

MDM-REQ-009 A record whose `classification` is above INTERNAL shall reference a DistributionControl whose `maxClassification` is at least the record's classification. The JSON Schema enforces the presence of the reference; rule BR-012 (§9.9) checks the authorisation.

MDM-REQ-010 Registry values that are not codes of the target vocabulary shall not be coerced silently. The record shall carry the nearest canonical code, the original value in `provenance.legacyValue`, and the conflict shall be recorded as a BaselineIssue (TDS-REQ-035, TDS-REQ-044). In MDM 1.3 this applies to the AIDCF-007A v1.1 registry status `PROPOSED`, which is not a RecordLifecycle code: registry records are imported with `status` CANDIDATE.

6 Identifier rules

Canonical codes are carried both in the IRI (for registry objects) and as typed properties so that JSON consumers do not parse IRIs.

Code	Pattern	Property
Domain	one of the 15 codes GOV ... EVA	`domainCode`
Family	`DOM-0N` (N 1 to 6)	`familyCode`
Control	`MCF-DOM-NNN` (001 to 024)	`controlCode`

Code	Pattern	Property
Critical gate	`CG-NN` (01 to 15)	`gateCode`
Gate criterion	`GC-NN-N`	`criterionId`
Release	`AIDCF-REL-YYYY.MM.N`	`releaseId`, `releaseRef`
Baseline issue	`BI-NNN`	`issueCode`
Artifact	as `semanticOwner`	`semanticOwner`, `sourceArtifact`

MDM-REQ-011 A record shall never embed a version number in its identifier, except release identifiers and the ontology version IRI (TDS-REQ-029). A change of meaning creates a new identifier and a SUPERSEDES edge; the superseded record is retained with status SUPERSEDED.

MDM-REQ-012 External keys (asset tags, CMDB keys, ticket numbers, AI Trust Graph identifiers) shall be recorded as `externalIdentifiers` pairs (`identifierScheme`, `identifierValue`) and never used as `id` (TDS-REQ-031).

MDM-REQ-013 Relationship-record identifiers shall be stable for the same edge across rebuilds of the same release. The AIDCF-015 reference implementation derives them from a digest of source, property, target and, for PROPAGATES_TO, the scenario; any equivalent deterministic scheme is conformant.

7 Classes and vocabularies

7.1 Canonical classes

The 48 classes of AIDCF-014 §7 are declared as OWL classes with their AIDCF-014 definitions (`skos:definition`), owner (`aidcf:semanticOwnerArtifact`) and chain step (`aidcf:chainStep`), grouped under abstract superclasses that are not instantiated (AidcfRecord, GovernanceRecord, CapabilityElement, ArchitectureRecord, ArchitectureElement, FailureRecord, AdverseCondition, ControlFrameworkElement, EvidenceElement, AssessmentElement, AssuranceChallenge, DecisionElement, ReportingElement, ContinuingAssuranceElement).

Chain step	Classes (owner)
Context	ReleaseManifest, BaselineIssue, ValidationResult, MigrationRecord, DistributionControl, Event (AIDCF-014); OperatingState (AIDCF-003); IntegrationHandoff (AIDCF-005)
1 Decision question	AssuranceProgram (AIDCF-011); Assessment (AIDCF-009)
2 to 4	CapabilityInstance, ServiceOutcome, RequiredFunction (AIDCF-003)
5 Architecture	ArchitectureSnapshot (AIDCF-013); InfrastructureSystem, AssetInstance, Interface, Resource, CapacityAssertion (AIDCF-003)
6 Dependency and fault domain	Dependency, FaultDomain (AIDCF-003)
7 Threat, hazard, scenario	Threat, Hazard, FailureMode, FailureScenario (AIDCF-004)
8 Safeguard	Safeguard (AIDCF-004)
9 Control	ControlDomain, ControlFamily, Control (AIDCF-007A)
10 and 11 Evidence	EvidenceRequirement, EvidenceObject, EvidencePackage (AIDCF-008)

Chain step	Classes (owner)
12 Control Assessment	AssessmentProcedure, Sample, TestRecord, ControlAssessment (AIDCF-009)
13 Finding and UNKNOWN	Finding (AIDCF-009); UnknownRecord (AIDCF-008)
14 to 16	CriticalGate, GateResult, ScoreSnapshot, DecisionRecommendation, AssuranceDecision (AIDCF-010); ReviewRecord (AIDCF-009)
Loop	AssuranceStatement, AssuranceReport (AIDCF-012); Action, ReassessmentTrigger (AIDCF-011)

Each class has a profile of required and optional properties that realises AIDCF-014 §8A. The model holds the profiles; the generated class schema and the class SHACL shape enforce the same required list.

7.2 Reference classes

AIDCF-014 §7 uses three AIDCF-003 reference classes in relationship ranges without profiling them. MDM 1.3 declares them with a minimal profile under the abstract superclass ReferenceEntity:

Class	Profile	Used by
Location	`locationKind` (SITE, BUILDING, HALL, ROOM, ROW, RACK_POSITION, LOGICAL)	LOCATED_IN
PartyEntity	`partyKind` (ORGANISATION, ROLE, ACTOR, TEAM)	OPERATED_BY, CONTROLLED_BY, DEPENDS_ON (external parties), owner, assessor, reviewer and authority properties
Component	`componentKind`	COMPOSED_OF (asset parts)

7.3 Supporting classes pending AIDCF-014 registration

AIDCF-003 v1.1 models industrial automation and control system security with the IEC 62443 series and introduces Assertion as the unit to which evidence attaches. These concepts are needed to serialise CG-07 and CG-15 evidence but are not among the 48 AIDCF-014 classes. MDM 1.3 declares them as supporting classes, each annotated `aidcf:registrationStatus` "pending AIDCF-014 registration" and each with `x-aidcf-registration: supporting` in its schema:

Class	Superclass	Profile (required)	Notes
IACS	InfrastructureSystem	system profile plus `controlledFunctions`; optional `assetOwner`, `integrator`, `maintainer`	The control systems of power, cooling, fire and building plant, treated as systems so that every architecture relationship applies.
IACSZone	InfrastructureSystem	system profile plus `riskAssessmentRef`, `boundaryDescription`, `controlledFunctions`	A zone as defined by a security risk assessment; not a physical security zone.
Conduit	Interface	`interfaceKind`, `channels`, `remoteAccess`, `countermeasures`	Connects two or more zones via CONNECTS_TO; access paths shall be classified at least CONFIDENTIAL (MDM-R12).

Class	Superclass	Profile (required)	Notes
SecurityLevel	SupportingRecord	`securityLevelKind` (SL-T target, SL-C capability, SL-A achieved), `subjectRef`, `slVector` (`fr1` to `fr7`, each 0 to 4), `basis`	Assertion-bearing. An SL-A that has not been assessed is recorded as an UNKNOWN Record against the zone, not as a SecurityLevel with a guessed vector.
FoundationalRequirement	SupportingRecord	`frCode` (FR1 to FR7)	Reference records; the labels follow AIDCF-003 v1.1 section 72.
Assertion	SupportingRecord	`subjectRef`, `predicate`, `assertionConditions`, `confidence`; optional `objectRef`, `objectValue`	A testable proposition with scope and conditions; assertion-bearing; may be listed in ArchitectureSnapshot `assertionRefs`.

MDM-REQ-014 A producer may use the supporting classes. A consumer that does not support them shall accept records of these classes as opaque records (envelope only) and shall not reject a dataset because they are present.

MDM-REQ-015 When AIDCF-014 registers a supporting class, the next schema set shall move it to `canonical` without changing its IRI. If AIDCF-014 registers it with a different name, the next schema set shall keep the MDM IRI as a deprecated equivalent class (`owl:deprecated true`, `owl:equivalentClass`) and provide a MigrationRecord.

AIDCF-014 §9 lists the range of SUPPORTS explicitly (CapacityAssertion, Dependency, FaultDomain, ControlAssessment, GateResult, FailureScenario) and does not include Assertion, whereas AIDCF-003 §72 and its relationship table expect evidence to support an Assertion. MDM 1.3 includes Assertion in the assertion-bearing set used for the CHALLENGES and QUALIFIES ranges, keeps the AIDCF-014 SUPPORTS range unchanged, and records the question in Annex A.

7.4 Auxiliary class

RelationshipRecord (owner AIDCF-014) carries the AIDCF-014 §10 relationship envelope for one edge. It is not one of the 48 classes and has no chain step. Profile: `relationshipType`, `sourceRef`, `targetRef`, `confidence`; optional `operatingStates`, `dependencyRef`, `faultDomainRef`, `scenarioRef`, `stepOrdinal`, `gateRole`, `constraintType`, `migrationRef`, `edgeEvidenceRefs`.

7.5 Assertion-bearing and assessable classes

Assertion-bearing classes (require `reviewState`; range of CHALLENGES and QUALIFIES): CapacityAssertion, Dependency, FaultDomain, ControlAssessment, GateResult, FailureScenario, Finding, UnknownRecord, Event, Assertion, SecurityLevel, RelationshipRecord.

Assessable classes (require `scope`): Assessment, ControlAssessment, GateResult, Finding, UnknownRecord, ScoreSnapshot, DecisionRecommendation, AssuranceDecision, AssuranceStatement, AssuranceReport, RelationshipRecord.

7.6 Controlled vocabularies

Every coded property is bound to one scheme. Codes are read from `registries/vocabularies/*.csv` when the registry defines them, and are otherwise declared in the model with the AIDCF artifact that defines them:

Source	Schemes
`registries/vocabularies/*.csv`	ClaimType, AssuranceLevel, ControlResult, ImplementationMaturity, GateState (`gate-results.csv`), DecisionState, DecisionLifecycle, Confidence and ConfidenceFlag (`confidence.csv`, split by `kind`), EvidenceClass, QualityDimension, QualityBand, Sufficiency, IntegrityLevel, ControlProfileTier, EvidenceConformanceLevel, UnknownType, UnknownSeverity, OperatingState, AssessmentMethod, ControlType, LifecycleStage, ScoreBand
AIDCF-014	RecordLifecycle, AssertionState, Classification, EventType, SourceHealth, EventSeverity, Materiality, HumanDisposition, ChangeType, BaselineStatus, ReviewType, ApprovalOutcome, ConstraintType, RelationshipType (the 41 codes of §9)
Other AIDCF artifacts	Domain, GateRole, GateApplicability, Criticality, WorkloadClass, ReconciliationStatus, DependencyType, DependencyStrength, SafeguardFunction, Activation, HazardType, FindingClass, FindingStatus, ReviewIndependence, ReviewOutcome, ReportClass, ActionStatus, TriggerState, HandoffType, MappingType, SecurityLevelKind, FoundationalRequirementCode, PartyKind, LocationKind
AIDCF-016	CriterionOutcome (MET, NOT_MET, UNKNOWN, NOT_EVALUATED), UnknownStatus (OPEN, DISPOSITIONED, RESOLVED), ProvenanceMethod, SignatureStatus, ValidationLayer, ValidationOutcome, Weighting

MDM-REQ-016 For every scheme sourced from a registry file, the JSON Schema enumeration and the OWL SKOS concept scheme shall contain exactly the codes of that file. A release shall verify this (VOCABULARY layer, §11).

MDM-REQ-017 Legacy codes in `control-result-aliases.csv`, `decision-aliases.csv` and `relationship-aliases.csv` shall be accepted only on input and mapped to the canonical code before validation; they appear in the ontology only as `skos:hiddenLabel` of the canonical concept. A legacy code with no canonical mapping (for example the decision alias "NOT ASSESSED") shall not be mapped to any decision state.

8 Relationships and structural rules

8.1 Forward edges

Each of the 41 AIDCF-014 §9 relationships is an OWL object property named in lowerCamelCase (`HAS_OUTCOME` → `aidcf:hasOutcome`) with the AIDCF-014 domain and range expressed as class unions, annotated with `aidcf:relationshipCode`, `aidcf:cardinalityOut`, `aidcf:cardinalityIn` and, where §10 requires them, `aidcf:qualifierProperty`. In JSON an edge is an array of target identifiers on the source record.

MDM-REQ-018 Every edge shall be serialised on its source record in the forward direction defined by AIDCF-014 §9. Inverse properties (for example `aidcf:assesses`, inverse of `assessedBy`) are declared in OWL for navigation and query but shall not be asserted in records (MDM-R11). The JSON Schemas reject them as undeclared properties and the SHACL shapes reject them with `sh:maxCount 0`.

MDM-REQ-019 SHARES_FAULT_DOMAIN_WITH is symmetric and shall be asserted once per pair, from either member. Consumers shall treat it as holding in both directions.

8.2 Relationship records

AIDCF-014 TDS-REQ-041 makes every edge a first-class record. MDM 1.3 serialises both projections: the forward edge on the source record (for graph traversal and SHACL) and a RelationshipRecord (for the envelope, qualifiers, conditions, review state, classification and evidence of the edge).

MDM-REQ-020 The set of asserted edges and the set of RelationshipRecords shall be in bijection: every RelationshipRecord shall correspond to an asserted edge with the same source, relationship type and target, and every asserted edge shall have at least one RelationshipRecord (MDM-R10). PROPAGATES_TO may have one RelationshipRecord per scenario step that uses the same edge.

MDM-REQ-021 RelationshipRecords shall carry the qualifiers and conditions required by AIDCF-014 §9 and §10:

Relationship	Mandatory qualifiers and conditions
DEPENDS_ON	`dependencyRef` to the Dependency record whose `sourceRef` and `targetRef` equal the edge; `operatingStates`
SHARES_FAULT_DOMAIN_WITH	`faultDomainRef` to a FaultDomain whose members include both endpoints
PROPAGATES_TO	`scenarioRef`, `stepOrdinal` (1 to n, in the order of the scenario's `propagationSteps`); `operatingStates`
PROTECTED_BY, RECOVERS_THROUGH	`operatingStates`
EVALUATES_GATE (Control to CriticalGate)	`gateRole` equal to the role in `registries/controls/gate-mappings.csv`
CONSTRAINS	`constraintType` (CEILING, CONDITION, LIMITATION, BAND_WITHHELD, GATE_EFFECT)
MIGRATED_FROM	`migrationRef` to the MigrationRecord that produced the source

MDM-REQ-022 A FailureScenario's `propagationSteps` shall be an ordered list (`@list`) of exactly the PROPAGATES_TO RelationshipRecords whose `scenarioRef` is that scenario.

8.3 Multi-valued properties

JSON distinguishes an empty array from an absent property; RDF does not. A required multi-valued property (for example `gateMappings` on a control with no gate mapping, or `controlEvaluations` on a meta-gate with no mapped control) may therefore be an empty array in JSON and has no triples in RDF.

MDM-REQ-023 JSON Schemas shall enforce the presence of required multi-valued properties and any explicit minimum item count. SHACL shapes shall enforce only the explicit minimum counts of multi-valued properties (for example `gateResultRefs` at least 15 on an AssuranceDecision), never a minimum of one derived from "required" alone.

8.4 Cardinality

Out-cardinality and in-cardinality of AIDCF-014 §9 are enforced in SHACL with property shapes on the source class (`sh:path`) and on the target class (`sh:inversePath`), including the class-specific cardinalities (for example GateResult EVALUATES_GATE exactly one CriticalGate; ControlFamily COMPOSED_OF-in exactly one ControlDomain; Control COMPOSED_OF-in exactly one ControlFamily; Assessment GOVERNED_BY exactly one AssuranceProgram; AssuranceDecision APPROVED_BY at least one ReviewRecord).

8.5 Review and approval cardinality

AIDCF-014 §9 gives REVIEWED_BY and APPROVED_BY each an in-cardinality of 1..* on ReviewRecord. Read literally, every ReviewRecord would need both an incoming REVIEWED_BY and an incoming APPROVED_BY. MDM 1.3 applies the in-cardinality by review type:

MDM-REQ-024 A ReviewRecord of `reviewType` REVIEW shall be the target of at least one REVIEWED_BY edge; a ReviewRecord of `reviewType` APPROVAL shall be the target of at least one APPROVED_BY edge. APPROVED_BY shall target only APPROVAL records and REVIEWED_BY only REVIEW records (MDM-R17).

8.6 Classification of edges

MDM-REQ-025 A RelationshipRecord shall be classified at least as high as the higher of its two endpoints (TDS-REQ-065), and records describing OT access paths (Conduit) or security weaknesses (Findings against SEC or SUP controls) shall be classified at least CONFIDENTIAL (TDS-REQ-066) (MDM-R12).

9 Canonical decision rules

The rules in this section are computed, not entered. A conformant implementation shall reproduce them exactly from the records and the registry copies; recorded values that differ are validation failures. The AIDCF-015 v1.3 reference implementation provides an executable statement of every rule in `tools/aidcf_rules.py`.

9.1 Control results and evidence

The control result vocabulary is that of `registries/vocabularies/control-results.csv`. A result is valid only for the claim type assessed.

MDM-REQ-026 A ControlAssessment shall not be CONFORMING unless its `evidenceSufficiency` is at or above the `sufficiencyFloor` of the control's Evidence Requirement for the control's claim type and its evidence includes every `requiredClasses` code of that requirement (MDM-R07). Insufficient evidence makes the result UNKNOWN, with an UNKNOWN Record; it never reduces a score by a multiplier.

MDM-REQ-027 The assessed claim type shall be at least the control's claim type (BR-003), and the control's `requiredAssuranceLevel` shall equal the minimum assurance level of its claim type in `claim-types.csv` (MDM-R08).

9.2 Gate roll-up

The gate roll-up is SPEC 6.2 expressed over records. Inputs for a gate G:

- **Roles**: the CRITICAL and SUPPORTING controls of G in `registries/controls/gate-mappings.csv` (not domain lists, not implementation choice).
- **Control results**: for each mapped control, the result of its ControlAssessment, or NOT_ASSESSED where none exists.
- **Criteria**: one CriterionResult per criterion of G in `registries/gates/critical-gate-criteria.csv`, with outcome MET, NOT_MET, UNKNOWN or NOT_EVALUATED.
- **Gate evidence**: the GateResult `evidenceSufficiency` against the gate `evidenceFloor`.
- **UNKNOWNs**: UNKNOWN Records attached to the GateResult by HAS_UNKNOWN or CONSTRAINS and not RESOLVED.
- **Supporting evaluations**: `gateLevelEvaluation` text on SUPPORTING control evaluations.

The state is the first that applies:

Order	State	Condition
1	APPROVED_NA	`gateApplicability` is APPROVED_NA, with `naRationale` and a REVIEWED_BY review. Otherwise a claimed APPROVED_NA is UNKNOWN.
2	FAIL	Any CRITICAL control is PARTIAL or NONCONFORMING, or any criterion is NOT_MET.
3	NOT_ASSESSED	No CRITICAL control has a result other than NOT_ASSESSED, no criterion has an outcome other than NOT_EVALUATED, and no US4 or US5 UNKNOWN is attached (§9.3).
4	UNKNOWN	Any CRITICAL control is UNKNOWN or NOT_ASSESSED; or any criterion is UNKNOWN or NOT_EVALUATED; or gate evidence is below the floor; or a US4 or US5 UNKNOWN is attached; or a SUPPORTING control is NONCONFORMING without a gate-level evaluation; or (for CG-01 to CG-10 and CG-13 to CG-15) the gate has no CRITICAL control in the registry.
5	PASS	Otherwise: every CRITICAL control CONFORMING, every criterion MET, evidence at or above floor, no US4 or US5 attached, every NONCONFORMING SUPPORTING control evaluated.

PARTIAL never yields PASS. There is no conditional gate state.

MDM-REQ-028 A GateResult shall record `gateState`, `controlEvaluations` and `criteriaResults` that reproduce the roll-up exactly (MDM-R01). `controlEvaluations` shall list every registry CRITICAL control of the gate (NOT_ASSESSED, without `controlAssessmentRef`, where unassessed) and every assessed SUPPORTING control, each with its registry role and the result of its ControlAssessment, and each assessed input shall also be linked by EVALUATES_GATE from its ControlAssessment to the GateResult (MDM-R02).

MDM-REQ-029 `criteriaResults` shall contain exactly one entry per registry criterion of the gate. A criterion may be MET only when demonstrated with evidence at or above the gate floor.

9.3 Scope and NOT_ASSESSED

SPEC 6.2 applies to "in-scope CRITICAL controls". MDM 1.3 makes scope explicit:

MDM-REQ-030 Every registry CRITICAL control of an applicable gate is in scope for that gate. An implementation shall not narrow a gate's CRITICAL set to the controls it happened to assess; the only way to remove a gate from scope is APPROVED_NA with rationale and review, and the only way to remove a control from scope is a NOT_APPLICABLE ControlAssessment with rationale and review (BR-002).

The gate-results vocabulary defines NOT_ASSESSED as "applicable gate not yet evaluated". A gate on which no CRITICAL control has been assessed, no criterion evaluated and no critical UNKNOWN raised has not been evaluated; once any of these exists, SPEC 6.2 applies and the gate is FAIL, UNKNOWN or PASS. NOT_ASSESSED and UNKNOWN apply the same ceiling (§9.5), so the distinction changes reporting, not the decision.

MDM-REQ-031 A NOT_ASSESSED gate shall apply its `unknownCeiling` to the decision. NOT_ASSESSED shall never be treated as PASS or as APPROVED_NA.

9.4 Gate ceilings derived from AIDCF-010 text

`registries/gates/critical-gates.csv` states each gate's FAIL and UNKNOWN ceilings as prose, often with qualifiers (for example a ceiling for some outcomes and a different one for the decision as a whole). The decision ceiling (§9.5) needs one code per gate state. MDM 1.3 derives it deterministically:

1. If the text states a decision code "for the decision as a whole" or "for the whole decision", that code.
2. Otherwise, if the text starts with a decision code followed by "." or ";" (an unqualified leading code), that code.

3. Otherwise, the lowest-ranked decision code mentioned anywhere in the text (conservative).
4. If the text mentions no decision code, NOT_ASSURED (conservative default).

Decision ranks follow the AIDCF-010 order (Gate ceilings; derivation pseudocode `ORDER`): REASONABLY_ASSURED 5, CONDITIONALLY_ASSURED 4, LIMITED 3, INCONCLUSIVE 2, NOT_ASSURED 1. Every state has a distinct rank. NOT_ASSURED is the most restrictive state because it records that a controlling gate is demonstrated not met; INCONCLUSIVE records that no determination can be supported (`registries/vocabularies/decision-states.csv`).

MDM-REQ-032 Each CriticalGate record shall carry the registry prose (`failCeilingText`, `unknownCeilingText`), the derived codes (`failCeiling`, `unknownCeiling`) and the rule applied (`ceilingDerivation`). A consumer shall recompute the codes from the prose and reject a record whose codes differ (MDM-R13).

MDM-REQ-033 The derivation is a baseline accommodation, not a change to AIDCF-010. It shall be recorded as a CONDITIONALLY_BASELINED BaselineIssue in every release until AIDCF-010 publishes machine ceilings, at which point the published codes replace the derivation.

For the AIDCF-010 text copied into MDM 1.3 the rule yields:

Gate	FAIL ceiling	UNKNOWN / NOT_ASSESSED ceiling
CG-01	INCONCLUSIVE	INCONCLUSIVE
CG-02	INCONCLUSIVE	LIMITED
CG-03	LIMITED	LIMITED
CG-04	LIMITED	LIMITED
CG-05	LIMITED	INCONCLUSIVE
CG-06	NOT_ASSURED	LIMITED
CG-07	NOT_ASSURED	LIMITED
CG-08	NOT_ASSURED	LIMITED
CG-09	LIMITED	LIMITED
CG-10	LIMITED	LIMITED
CG-11	INCONCLUSIVE	LIMITED
CG-12	INCONCLUSIVE	LIMITED
CG-13	LIMITED	LIMITED
CG-14	LIMITED	LIMITED
CG-15	LIMITED	LIMITED

9.5 Decision ceiling and positive decisions

The decision ceiling is the minimum, by rank, of:

- REASONABLY_ASSURED (no constraint);
- the `failCeiling` of every FAIL gate and the `unknownCeiling` of every UNKNOWN or NOT_ASSESSED gate;
- for every UNKNOWN Record in scope that is not RESOLVED: US5 → INCONCLUSIVE, US4 → LIMITED, US3 → CONDITIONALLY_ASSURED (SPEC 5.7).

Ranks are distinct, so the minimum is a single code and no tie-break applies. A FAIL gate whose ceiling is NOT_ASSURED keeps the decision at NOT_ASSURED whatever INCONCLUSIVE ceilings (a US5 UNKNOWN,

a gate UNKNOWN ceiling or the AIDCF-010 coverage step) also apply: a demonstrated gate failure is not softened by undetermined positions. The INCONCLUSIVE ceilings remain recorded as limitations and reassessment conditions.

MDM-REQ-034 A DecisionRecommendation shall record the computed `decisionCeiling` and the `ceilingDrivers` that produce it, and its `recommendedState` shall not exceed the ceiling (MDM-R06). An AssuranceDecision shall not exceed the ceiling of any FAIL, UNKNOWN or NOT_ASSESSED gate (MDM-R05) or of any open UNKNOWN severity (MDM-R04), and shall reference every open US3 to US5 UNKNOWN in `unknownRefs`.

MDM-REQ-035 An UNKNOWN blocks a positive decision. A decision of REASONABLY_ASSURED or CONDITIONALLY_ASSURED requires every one of the fifteen Gate Results to be PASS or APPROVED_NA and no open US4 or US5 UNKNOWN; REASONABLY_ASSURED additionally requires no open US3 UNKNOWN; CONDITIONALLY_ASSURED with an open US3 requires a decision condition referencing it (MDM-R03, MDM-R04).

MDM-REQ-036 An AssuranceDecision shall reference exactly one Gate Result for each of CG-01 to CG-15, and shall be approved by an APPROVAL ReviewRecord whose reviewer is the `decisionAuthority` of the AssuranceProgram (MDM-R17).

MDM-REQ-037 An AssuranceStatement or AssuranceReport shall state the decision state of the decision it references, never a stronger state, and shall not be valid beyond the decision's `effectiveTo` (BR-007).

9.6 Evidence requirements of gates

AIDCF-014 HAS_EVIDENCE_REQUIREMENT allows CriticalGate as a source, and the Evidence Requirement profile requires a claim type, but `critical-gates.csv` states only required classes and a floor. MDM 1.3 gives each gate one Evidence Requirement with the gate's `requiredClasses` and `evidenceFloor`, and a `claimType` equal to the highest claim type among the registry controls mapped to the gate. CG-11 and CG-12 have no mapped controls and use DESIGN pending an AIDCF-010 statement. This is recorded in Annex A.

9.7 Score

The score method is SPEC 5.12. MDM 1.3 fixes the computation so that it can be reperformed:

- Inputs: one ScoreInput per registry control (360 in AIDCF-007A v1.1): `controlRef`, `controlResult` (the ControlAssessment result, or NOT_ASSESSED), `controlAssessmentRef` where assessed, and `scoreValue` CONFORMING 100, PARTIAL 50, NONCONFORMING 0, absent otherwise.
- Coverage: `coveragePercent` = scored ÷ (in scope − NOT_APPLICABLE) × 100, with `inScopeCount`, `notApplicableCount`, `assessedCount`, `scoredCount`, `unknownCount` and `notAssessedCount`.
- Mean: mean of the scored values (UNWEIGHTED unless an approved profile weights them).
- Band: B5 ≥ 90, B4 ≥ 75, B3 ≥ 50, B2 ≥ 25, otherwise B1; WITHHELD if coverage is below 80% or any US4 or US5 UNKNOWN is open in scope, with `bandWithheldReason`.
- Rounding: coverage and means to four decimal places.
- Domain scores: the same computation over each domain's controls.
- Hash: `calculationHash` = SHA-256 of the canonical JSON `{"method": "AIDCF-016 MDM score 1.3", "weighting": <weighting>, "inputs": [[controlId, result, value or null], ...]}` with inputs sorted by control identifier, keys sorted and separators "," and ":".

MDM-REQ-038 A ScoreSnapshot shall carry all inputs, the method and the hash, and a consumer shall recompute coverage, mean, band, domain scores, UNKNOWN overlay and hash from the inputs and from the ControlAssessments; any difference is a failure (SCORE layer). Scores never override gates.

9.8 Change impact

AIDCF-014 §22 requires a change event to identify every dependent record. MDM 1.3 fixes the traversal so that BR-013 and TR-17 can be checked. Starting from the event's `subjectRef`, a record X is affected if:

1. X DEPENDS_ON, IMPLEMENTED_BY, COMPOSED_OF, PROTECTED_BY, RECOVERS_THROUGH, REQUIRES_FUNCTION or EVIDENCED_BY an affected record, or SHARES_FAULT_DOMAIN_WITH it in either direction; or X is a FaultDomain with an affected member; repeated to a fixed point, without expanding through the context containers ArchitectureSnapshot, CapabilityInstance, Assessment and AssuranceProgram;
2. then X is a ControlAssessment whose `subjectRef` or evidence is affected, the Control it assesses, a GateResult it is an input to, a Control governing an affected Safeguard, or an AssuranceDecision that references an affected GateResult.

MDM-REQ-039 A MATERIAL Event shall list in `affectedAssertions` exactly the records produced by this traversal (BR-013). Every affected Control, ControlAssessment, GateResult, AssuranceDecision and ServiceOutcome shall be the target of TRIGGERS_REASSESSMENT from the event or one of its candidate triggers, or carry INVALIDATED_BY to the event (TR-17).

MDM-REQ-040 While an Event's `humanDisposition` is PENDING, the event shall have `reviewState` CANDIDATE, every RelationshipRecord created by it shall be CANDIDATE, and no record shall have been SUSPENDED or WITHDRAWN because of it (TDS-REQ-062, BR-011).

9.9 Rule catalogue

Rules are executed in the BUSINESS_RULE layer (§11). BR-001 to BR-015 are the AIDCF-014 §18 rules; MDM-R01 to MDM-R20 are defined here. BR-015 (evidence reused across claim types) is reported as a warning requiring a fresh SUPPORTS review.

Rule	Check
MDM-R01	Gate state equals the §9.2 roll-up; `gateCode` equals the evaluated gate; APPROVED_NA has rationale and review.
MDM-R02	`controlEvaluations` list every registry CRITICAL control and every assessed SUPPORTING control with registry role and ControlAssessment result; inputs linked by EVALUATES_GATE; `criteriaResults` list exactly the registry criteria; no input is mapped to a gate it is not mapped to in the registry.
MDM-R03	Positive decisions blocked as MDM-REQ-035; exactly one Gate Result per gate.
MDM-R04	Decision within every open UNKNOWN severity ceiling; open US3 to US5 referenced; US3 condition for CONDITIONALLY_ASSURED.
MDM-R05	Decision within every FAIL, UNKNOWN and NOT_ASSESSED gate ceiling.
MDM-R06	Recommendation ceiling equals the §9.5 computation; recommendation and decision within it.
MDM-R07	CONFORMING requires evidence at or above floor and every required class.
MDM-R08	Required assurance level consistent with claim type.
MDM-R09	An open US4 constrains a Gate Result that is UNKNOWN or FAIL; no PASS with an open US4 or US5 attached.

Rule	Check
MDM-R10	Edge and RelationshipRecord bijection; qualifiers and conditions as MDM-REQ-021; Dependency, fault-domain, scenario and gate-role consistency; propagation steps ordered 1 to n.
MDM-R11	No inverse property asserted.
MDM-R12	Edge classification at least that of its endpoints; OT access paths and security weaknesses at least CONFIDENTIAL.
MDM-R13	Registry fidelity: 15 domains, 90 families, 360 controls, 15 gates; control attributes, gate mappings, EVALUATES_GATE edges, Evidence Requirements, gate floors and ceiling prose equal the registry copies; machine ceilings follow §9.4.
MDM-R14	COMPOSED_OF only between permitted whole-part pairs and acyclic; SUPERSEDES same class and acyclic.
MDM-R15	GOVERNED_BY only between permitted pairs (Safeguard to Control; ControlAssessment to AssessmentProcedure or Control; Assessment to AssuranceProgram; any record to DistributionControl).
MDM-R16	EVALUATES_GATE only Control to CriticalGate, ControlAssessment to GateResult, GateResult to CriticalGate.
MDM-R17	APPROVED_BY targets APPROVAL records; REVIEWED_BY targets REVIEW records; decisions approved by the programme decision authority.
MDM-R18	The `subjectRef` of every UNKNOWN Record carries HAS_UNKNOWN to it.
MDM-R19	A DISPOSITIONED UNKNOWN has a disposition and due date; an OPEN US4 or US5 is reported as a warning (CG-12).
MDM-R20	Every record's `releaseRef` is the release of the manifest it is published with.

10 Canonical queries

AIDCF-014 §19 lists twelve canonical questions (Q01 to Q12). A conformant implementation at T4 or above answers them in SPARQL 1.1 over the JSON-LD projection with the ontology loaded.

MDM-REQ-041 Each canonical query shall be published as a `.rq` file with its expected result for the reference dataset, and the GRAPH_QUERY layer shall execute it and compare the result set (order-insensitive unless the query orders it) with the expected result. Expected results shall be derived from the dataset design or the registry, not captured from the query being tested.

The AIDCF-015 v1.3 queries answer the §19 questions for the reference dataset as follows:

Query	Reference-dataset question
Q01	Elements implementing the functions required by CI-001, with their Dependency records
Q02	Scenarios that defeat SO-01, the fault domains their steps touch, and the pre-state
Q03	Evidence, results and findings for MCF-SEC-005 by claim type and scope
Q04	Assertions supported or challenged by EV-011 and the decisions that depend on them
Q05	For CG-07: state, ceiling, CRITICAL controls, criteria not met and UNKNOWNs
Q06	For AD-001: state, score, approval, gate and UNKNOWN counts
Q07	Statements and reports against their decision
Q08	Gate Results affected by EVT-001

Query	Reference-dataset question
Q09	Providers relied on by two or more Dependency records
Q10	Successors of superseded and migrated records, with the migration
Q11	Candidate events awaiting review, with materiality, severity and age
Q12	Records visible only under the highly restricted distribution control

Two further queries check invariants: Q13 (edges created by a pending event that are not CANDIDATE: none) and Q14 (Gate Results that PASS with a CRITICAL input that is not CONFORMING: none).

11 Validation layers

11.1 Layers

A validation run executes the layers below in order and reports, for each layer, the number of checks executed, the number failed and the number of warnings. A layer that cannot execute (for example because a file does not parse) is a failure, not a skip.

Layer	Checks
REGISTRY	Cardinality 15/90/360; unique control identifiers; family and domain consistency; one evidence profile per control with matching claim type; `gate-mappings.csv` consistent with the gate columns of `controls.csv`; valid roles; registry copies equal to their recorded digests.
SYNTAX	Every JSON file parses; every Turtle file parses; every JSON-LD record file expands with the pinned context.
SCHEMA	Every schema validates against the 2020-12 metaschema; every record and the manifest validate against their class schema.
SHACL	The data graph (records plus ontology, no inference) conforms to the shapes graph.
BUSINESS_RULE	BR-001 to BR-015 and MDM-R01 to MDM-R20 (§9.9).
TRACEABILITY	TR-01 to TR-18 (AIDCF-014 §11, interpreted per Annex A).
GRAPH_QUERY	§10 queries against expected results.
SCORE	§9.7 reperformance.
INTEGRITY	Evidence fixture digests and sizes; evidence-package digests; no orphan fixtures.
MANIFEST	§13 checks.
VOCABULARY	MDM-REQ-016 and MDM-REQ-017.
EXPORT	AIDCF-014 CT-09: an export at INTERNAL withholds every record and RelationshipRecord above INTERNAL and keeps no edge to a withheld record.
FIXTURE	Every conformance fixture (§12) produces its expected outcome for its stated reason.

MDM-REQ-042 A validation run shall exit with a failure status if any check in any layer fails, shall report honest counts per layer, and shall publish its results as ValidationResult records (validationLayer, ruleId, outcome, message, validator, executedAt). A ValidationResult produced by the author's own tooling shall say so in `validator`.

MDM-REQ-043 A validation run shall write only to an output location excluded from the release manifest, so that running the validator never changes a file whose digest the manifest records.

11.2 SHACL execution profile

SHACL-SPARQL forbids VALUES, MINUS and federated queries in constraints, so rank comparisons in MDM 1.3 shapes use nested IF expressions. Constraints that must examine every edge or every relationship record are written as graph-level constraints (focus node: the ontology IRI; value: the offending record), and per-type qualifier checks use SPARQL-based targets with core `sh:minCount` constraints. These are performance choices with the same results as per-node constraints.

MDM-REQ-044 SHACL validation shall run with the OWL ontology supplied as the ontology graph and with no inference, so that class targets follow `rdfs:subClassOf` and no state is inferred (TDS-REQ-051, TDS-REQ-052).

12 Conformance classes and test catalogue

12.1 Conformance classes

Class	Name	Requirements	AIDCF-014 level supported
MDM-C1	Serialisation	§3 to §7: envelope, identifiers, vocabularies; SYNTAX, SCHEMA and VOCABULARY layers pass	T0, T1, T3
MDM-C2	Graph	MDM-C1 plus §8; SHACL layer passes; relationship records in bijection	T2 (with TRACEABILITY), T4
MDM-C3	Decision	MDM-C2 plus §9; BUSINESS_RULE, SCORE and GRAPH_QUERY layers pass	T5
MDM-C4	Continuing assurance	MDM-C3 plus §9.8 and MDM-REQ-040 for every Event	T6
MDM-C5	Release	MDM-C4 plus §13 and the full test catalogue; MANIFEST, INTEGRITY, EXPORT and FIXTURE layers pass	T7 (when the run is independent and the manifest signed)

MDM-REQ-045 A conformance claim shall name the class, the release identifier, the schema set version, the test-suite digest and the date, and shall state whether the run was independent. A claim of MDM-C5 shall not be used to support an AIDCF-014 T7 claim unless the validation run was executed independently and the manifest is signed.

12.2 Fixtures

A fixture is a JSON file with `fixtureId`, `description`, `layer`, `expect` (PASS or FAIL), the reason (`expectedRule` and/or `expectedText`) and either a standalone `record` or a `patch` applied to a copy of the reference dataset. Fixtures live in `tests/valid`, `tests/invalid` (syntax, schema and SHACL) and `tests/business` (rules, score, integrity and manifest).

MDM-REQ-046 A negative fixture passes only if the run fails at the stated layer with the stated reason. A negative SHACL fixture also requires that the stated violation is absent from the unpatched dataset, so that a fixture cannot pass on a pre-existing defect.

12.3 Test catalogue

Test	AIDCF-014 test	Fixtures in AIDCF-015 v1.3
MDM-T01 Positive records	CT-01	V-001 to V-004 (capability, control assessment, gate result, relationship record); the full dataset (58 classes, 41 relationship types)
MDM-T02 Missing mandatory	CT-02	I-SCH-001 (restored v1.1 capability without owner), I-SCH-006, I-SCH-007, I-SCH-010; legacy v1.1 fixtures
MDM-T03 Closed structure	CT-02	I-SCH-002 (undeclared property), I-SCH-008 (inverse property), I-SCH-003 to I-SCH-005, I-SCH-009 (codes and patterns), I-SHA-005 (assurance level), I-SHA-009 (gate code)
MDM-T04 Syntax	—	I-SYN-001 (malformed JSON-LD), I-SYN-002 (broken Turtle)
MDM-T05 UNKNOWN handling	CT-03	I-SHA-003, I-SHA-010, I-SHA-012, B-004, B-005, B-007
MDM-T06 Gate dominance	CT-04	I-SHA-001, I-SHA-002, B-001 (PASS on PARTIAL CRITICAL inputs), B-002 (PASS with no CRITICAL control assessed), B-003, B-008, B-012
MDM-T07 Score reproducibility	CT-05	V-008, B-006, B-021, B-022
MDM-T08 Report consistency	CT-06	I-SHA-007, B-013, B-014
MDM-T09 Supersession and migration	CT-07, CT-08	Dataset CAP-CAP-01/02 and MIG-001/002; B-015
MDM-T10 Security	CT-09	I-SHA-008, EXPORT layer
MDM-T11 AITG handoff	CT-10	B-011
MDM-T12 Event impact	CT-11	B-009, B-016, B-018
MDM-T13 Anti-gaming	CT-12	B-017 (NOT_APPLICABLE without rationale), B-019 (self-closure), B-020 (registry tamper)
MDM-T14 Relationship records	—	I-SHA-004, I-SHA-006, I-SHA-011, B-010
MDM-T15 Evidence and manifest integrity	—	B-023 (evidence altered), B-024 (file altered after manifest), B-025 (signature claimed)
MDM-T16 Performance	CT-14	Full run time reported in `outputs/validation-results.json`

CT-13 (baseline conflict) is exercised by the REGISTRY layer and the BaselineIssue records; a fixture that creates a BLOCKED issue from conflicting registry counts is not yet provided.

13 Release manifest

The ReleaseManifest is the AIDCF-014 §4 record serialised as JSON with the envelope.

MDM-REQ-047 The manifest shall list every file of the release package except the excluded paths, each with `filePath`, `sha256` over the exact bytes, `byteSize` and `mediaType`; the excluded paths shall be the validation output location and the manifest itself, and shall be listed in `excludedPaths`.

MDM-REQ-048 `registryVersions`, `vocabularyVersions` and `schemaVersions` shall list each registry CSV, vocabulary CSV and schema-set file with version and digest; `knownIssues` shall reference every BaselineIssue of

the release; `migrationSet` every MigrationRecord; `testSuite` the suite version, a digest over the fixture files and the location of the validation results.

MDM-REQ-049 `integrity.manifestDigest` shall be the SHA-256 of the RFC 8785 canonical form of the manifest with `integrity.manifestDigest` removed. Until a detached signature exists, `integrity.signatureStatus` shall be PENDING and no `signaturePath` shall be given; every approval not yet given shall be listed with `approvalOutcome` PENDING, and `releaseStatus` shall be no higher than REVIEWED (TDS-REQ-022, TDS-REQ-026).

MDM-REQ-050 Every record of the release shall carry the manifest's `releaseId` as `releaseRef` (MDM-R20), and a consumer shall treat as UNKNOWN any record whose release, schema or vocabulary version is not listed in the manifest it declares (TDS-REQ-025).

14 Versioning and migration

MDM-REQ-051 The schema set version, the `schemaVersion` constant, the schema `$id` path, the context file name, the shapes IRI and the ontology version IRI shall change together. A record declares the schema set it conforms to through `schemaVersion`.

MDM-REQ-052 A new schema set shall not delete a class, property or code used by the previous set. It shall deprecate it (`owl:deprecated true` with a pointer to its replacement; TDS-REQ-054) and publish a MigrationRecord whose mappings carry `fromId`, `toId`, `changeType` (IDENTIFIER, VOCABULARY, RESULT_REINTERPRETATION, STRUCTURE, ADDED, REMOVED) and `rationale`.

MDM-REQ-053 A record migrated from a previous release shall carry MIGRATED_FROM to its predecessor identifier with the MigrationRecord as `migrationRef`. When a migration re-evaluates a result under a canonical rule (for example a gate that a previous release recorded as PASS on PARTIAL CRITICAL inputs), the mapping shall be RESULT_REINTERPRETATION and shall state the rule applied; the previous value shall not be overwritten in the previous release.

MDM 1.3 migration from AIDCF-015 v1.2 (MigrationRecord MIG-002 in the reference dataset): synthetic records move to the `urn:aidcf:syn:` namespace; evidence-class words (CONFIGURATION, OPERATIONAL, VALIDATION, DOCUMENTARY) become E-codes with the original kept in `provenance.legacyValue`; MCF-RES-017 changes from PARTIAL to UNKNOWN (evidence below floor, no E4 class); every Gate Result is recomputed under §9.2 with registry roles, each change recorded as a RESULT_REINTERPRETATION mapping from the v1.2 state (for example CG-03, CG-07 and CG-15 change from PASS to FAIL, and CG-02 from PASS to NOT_ASSESSED); the decision changes from LIMITED to NOT_ASSURED (controlled by the CG-07 FAIL ceiling).

15 Limitations of this edition

- The supporting classes of §7.3 await AIDCF-014 registration.
- Gate ceilings and gate evidence-requirement claim types are derived (§9.4, §9.6) pending machine values from AIDCF-010.
- The registry status code `PROPOSED` is outside the AIDCF-014 record lifecycle (MDM-REQ-010).
- w3id registration is pending; the IRIs do not resolve.
- The AIDCF-015 v1.3 reference run is self-validation; its manifest is unsigned and its approvals are pending. No MDM-C5 claim supporting T7 can be made from it.
- The RFC 8785 canonicalisation in the reference implementation handles JSON without floating-point numbers only; the manifest profile contains none.

Annex A Interpretations of AIDCF-014 recorded for confirmation

Item	AIDCF-014 text	MDM 1.3 interpretation
A.1 REVIEWED_BY and APPROVED_BY in-cardinality	Both 1..* on ReviewRecord (§9)	Applied by `reviewType` (§8.5).
A.2 SUPPORTS range	Excludes Assertion (§9)	Assertion is assertion-bearing for CHALLENGES and QUALIFIES; SUPPORTS to Assertion is not permitted until AIDCF-014 decides.
A.3 Scope of CRITICAL controls	"in-scope CRITICAL control" (SPEC 6.2)	Every registry CRITICAL control of an applicable gate is in scope (§9.3).
A.4 NOT_ASSESSED gate	Vocabulary only	Defined by the §9.2 order 3 condition; applies the unknownCeiling.
A.5 Gate ceilings	Prose (AIDCF-010)	Derived per §9.4.
A.6 Gate evidence requirement claim type	Not stated	§9.6.
A.7 TR-05 coverage	Every ABSOLUTE or MATERIAL dependency and every fault domain is covered by at least one failure scenario	A dependency is covered when its provider is the source of a PROPAGATES_TO step of some scenario; a fault domain when a member is a node of some scenario's steps.
A.8 TR-06 affected elements	Expected safeguards protect the affected elements	The affected elements are all nodes of the scenario's propagation steps; at least one is PROTECTED_BY an expected safeguard.
A.9 TR-09 scope	Every Evidence Requirement in scope has evidence of a required class or an UNKNOWN	Applies to the Evidence Requirements of assessed controls and of gates evaluated as PASS, FAIL or UNKNOWN.
A.10 TR-17 impact	Event impact identifies dependent records	§9.8 traversal; targets of classes Control, ControlAssessment, GateResult, AssuranceDecision and ServiceOutcome require TRIGGERS_REASSESSMENT or INVALIDATED_BY.
A.11 Registry status	§11A.1 codes	`PROPOSED` imported as CANDIDATE with the original value kept (MDM-REQ-010).

Final doctrine

A machine-executable model earns trust only by being executed. Every decision-relevant value in an AIDCF dataset — gate state, ceiling, decision, score, impact — is either taken unmodified from a pinned registry or recomputed by a published rule, and any recorded value that cannot be reproduced is a failure, however plausible it looks.